A Byte-Level Multilingual News Recommender for Microcontrollers

Abstract. Running a news recommender on the device itself keeps a reader's clicks private, but it runs into two difficulties. A new user has no history to learn from, and the usual models store a word-embedding table of tens to hundreds of megabytes that will not fit in a microcontroller's flash. We avoid the table by reading a title as its raw UTF-8 bytes: a small char-CNN with a 256-value vocabulary, distilled from a frozen multilingual sentence-transformer, so one sub-2 MB model reads all fourteen languages of xMIND without any per-language vocabulary. Around this encoder we search three families of architecture (unconstrained, microcontroller-constrained, and binary-aware) and evaluate each at full, 8-bit and 1-bit precision, measuring ranking quality on MIND against flash, RAM and an energy estimate. The 8-bit constrained model scores 0.610 AUC in 204 KB, matching a 27 MB NRMS baseline (0.607) at about one part in 130 of its size; lifting the size limit reaches 0.647 in 790 KB. A naive 1-bit model barely beats chance (0.52), but ReActNet-style training from a distilled start recovers it to 0.572 in 186 KB. Along the way we find that the distilled initialisation is worth +3.3 AUC, that results vary little between seeds (std below 0.012), that the embedding can be cut to 64 dimensions almost for free, and that the exported topic prior raises a cold user from chance (0.500) to 0.551. The multilingual figures are cross-lingual transfer through the content encoder, since the user side still learns from English clicks; measuring on real Pi 5 and STM32H7 boards is left for future work. Code and artefacts are released.

Keywords: Edge AI · Neural Architecture Search · Binarized Neural Networks · Multilingual News Recommendation · TinyML · Knowledge Distillation

1 Introduction

News apps increasingly run their personalisation on the phone or the sensor node rather than in the cloud, which is attractive for privacy: the record of what a user reads never leaves the device. Two difficulties come with that setting. A model that starts from nothing for each new user recommends poorly until it has seen enough clicks, and the standard neural recommenders (NRMS [14], NAML [13] and their relatives) represent a title with a word-embedding matrix. That matrix is large: GloVe-300d over 30–100k words is already 37–120 MB, one table per language, and none of it fits a microcontroller's few megabytes of flash.

We attack both problems offline, on MIND [16] and its translated extension xMIND [2], and then shrink the result for the device. The idea is to drop words and read a title as its UTF-8 bytes. A byte takes 256 values, so the vocabulary is a single 256-row table shared by every language, and one small convolutional encoder serves Chinese, Georgian and English alike. We train this student to reproduce the embeddings of a frozen multilingual teacher [11], so that a translated title lands near its English original.

We contribute three things. A byte-level content encoder removes the embedding-table memory wall and covers fourteen languages with a single model under two megabytes. Around it we benchmark three architecture searches (unconstrained, microcontroller-constrained and binary-aware) at three precisions, reporting accuracy alongside flash, RAM and energy. Finally we make the binary case, otherwise near chance, work: ReActNet activations and Bi-Real shortcuts started from the distilled weights, together with the seed-variance, embedding-truncation and cold-start checks a reader will want. We are not aware of an earlier byte-level, table-free news recommender examined jointly over architecture search and precision for microcontroller targets; the nearest on-device NAS and binary work is on vision [4, 5, 9], and existing multilingual recommenders [2] are built for servers.

2 Related Work

Neural news recommendation on MIND is well charted. NRMS [14], NAML [13], LSTUR, NPA and DKN encode titles and users with attention or knowledge graphs and rank the candidates of each impression by AUC, MRR and nDCG; their published test AUC on the large split runs from about 0.65 for DKN to 0.68 for NRMS [16]. Language-model recommenders (PLM-NR, Fastformer, UNBERT) do better, around 0.70–0.73 [15], but only by wrapping a BERT-sized encoder of several hundred megabytes, so they are out of reach on a microcontroller; we keep NRMS as the strongest baseline that is actually small. The tools we borrow are vocabulary-free character and byte models [1], cross-lingual distillation [11], memory-budgeted NAS [4], and the binary-network line of XNOR-Net [10], Bi-Real [7] and ReActNet [6]; what is new is putting them together for on-device multilingual news.

3 Method

We treat recommendation as impression-level ranking: given the titles a user has clicked and a set of candidates, score the candidates. Every title, clicked or candidate, becomes a vector through the same encoder. A frozen `paraphrase-multilingual-MiniLM-L12-v2` [12] supplies a 384-dimensional target for each English title, and because the xMIND titles are translations of those same articles we ask the student to map every translation to that one target, which is what makes the space language-agnostic. The student reads a title's bytes through a 256-entry embedding, a few depthwise-separable one-dimensional convolutions (cheap enough for a microcontroller), a masked average over positions, and a linear projection; a Matryoshka [3] term in the loss keeps the leading coordinates informative on their own. A light additive-attention encoder then summarises the user's history, and a candidate's score is the dot product of the two vectors, trained with one positive and a few negatives per step (AdamW).

The search has three arms that differ in what they may do. The first is unconstrained and simply looks for the most accurate encoder. The second must fit an aggressive on-device budget and is quantised to eight bits. The third must fit the same budget with one-bit weights. Each arm runs an evolutionary search over the channel count, depth and output width, scores a candidate by how well it distils, and rejects any that overrun the budget; precision is then the second axis. Quantisation is simulated with a straight-through estimator, per-channel for eight bits and sign-times-scale for one, and, following XNOR-Net, the byte embedding and the outermost layers stay in full precision. For the binary arm alone we add the ReActNet activations and Bi-Real shortcuts and start from the distilled weights, which is what makes one-bit training converge to something useful. Sizes and multiply-accumulates come from `thop`; the energy column multiplies the latter by the Horowitz 45 nm per-operation figures (4.6, 0.23, 0.0072 pJ for full, eight-bit and one-bit), read only as a relative ordering, since it ignores the memory traffic that often dominates on a real chip.

4 Experimental Setup

We use MINDsmall [16]: fifty thousand users, about 157k training and 73k development impressions over 51k articles, together with the fourteen NLLB translations of xMIND [2, 8]. xMIND provides only translated titles, keyed to the same article ids, so we reuse the English behaviour logs and swap the text. To be clear: every recommender we train, and the NRMS baseline, sees only English clicks, and the other languages appear only at evaluation. The multilingual numbers are therefore a test of cross-lingual transfer in the content encoder, not of native-language personalisation. We stay on the small split rather than the million-user MINDlarge because the full study — three arms, three precisions, the search and the fifteen-language evaluation — already takes 8.2 hours on one laptop; the small split has no public test set, so we evaluate on dev. The search runs a population of thirty over fifteen generations, scoring each candidate by a two-epoch distillation on twenty thousand titles and discarding budget violators; the thresholds (256 KB flash, 128 KB RAM, 2 M multiply-accumulates) are about eight times tighter than an STM32H7 can hold, so they bite at this scale. Everything runs on one laptop: an Intel Core Ultra 9 275HX with an eight-gigabyte RTX 5070 Laptop GPU and 32 GB of memory, PyTorch 2.12 on CUDA 13. The intended edge targets are a Raspberry Pi 5 and an STM32H743, but we did not have the boards, so their latency and energy remain future work and we report only laptop timings and the analytical estimate. The seed is fixed, the downloaded data is pinned by checksum, and the code, notebook, artefacts and a dependency lock are released.[1]

[1] `https://github.com/SepehrMohammady/MIND-Edge-Recommender`

Table 1. Architecture × precision on MINDsmall dev. **Bold**: best per group; †: each arm's native precision. The arm is the search that produced the architecture and the precision is the deployment quantisation, so off-diagonal cells are ablations. Energy is the Horowitz proxy.

Arm	Prec.	AUC	MRR	nDCG@10	Size (KB)	Energy (μJ)
NAS	FP32†	0.633	0.326	0.375	1566.8	168.7
NAS	INT8	**0.647**	**0.338**	**0.387**	789.8	4.65
NAS	Binary	0.588	0.297	0.342	563.1	4.25
Micro-NAS	FP32	0.605	0.307	0.352	266.8	15.90
Micro-NAS	INT8†	**0.610**	**0.314**	**0.360**	203.9	2.30
Micro-NAS	Binary	0.521	0.251	0.294	185.6	2.25
Bin. μNAS	FP32	0.593	0.290	0.336	311.4	15.43
Bin. μNAS	INT8	0.601	0.295	0.344	255.7	3.28
Bin. μNAS	Binary†	**0.546**	**0.273**	**0.317**	239.5	3.23

5 Results

Table 1 collects the nine combinations (the three arms found the architectures 256-4-384, 64-5-384 and 96-2-384). Two patterns emerge. Moving from full to eight-bit precision costs essentially nothing in accuracy — it sometimes helps, the usual quantisation-aware regularisation — while cutting the energy estimate by seven to thirty-six times and the size by a further one and a half to two. One bit is different: it loses five to nine AUC points, and because the embedding and outer layers stay in full precision its size advantage over eight bits is small at this scale. Measured on the laptop, a single title takes 0.34, 0.19 and 0.10 ms on the CPU through onnxruntime for the three architectures and under 0.7 ms on the GPU, well inside a 50 ms budget, with the CPU time tracking the multiply-accumulate count.

Our reproduced NRMS anchors the comparison: 92% of its 7.08 million parameters are the embedding table (27 MB, or 6.8 MB in eight bits at the same accuracy), and it reaches 0.607. The byte encoder matches that at 204 KB, thirty-three times smaller than even eight-bit NRMS, and beats it at 0.647 once the size limit is lifted. Across the fourteen languages the single eight-bit model ranges from 0.604 on English down to 0.491 on Georgian, strongest on Latin-script, higher-resource targets and weakest where the script is distinct and the translations poorer; Georgian at 0.491 is essentially chance, a reminder that this is transfer and not native personalisation.

Three smaller experiments round out the picture. Starting the encoder from the distilled weights rather than from scratch raises AUC from 0.600 to 0.633 at a fixed architecture, so the gain is from distillation, not capacity; this is the model we deploy. Over three seeds the eight-bit AUC moves by at most 0.012 (standard deviation 0.004), so the single-seed numbers are stable. Truncating the output from 384 to 64 dimensions barely changes its similarity to the teacher (0.454 to 0.459), so a memory-bound device can keep the first 64. And with a user's history blanked to mimic a newcomer the history-based score falls to chance (0.500), whereas ranking by the exported topic prior brings a cold user back to 0.551.

Sign-binarising the naive way leaves it near chance (0.521); with the ReAct-Net activations, the Bi-Real shortcuts and the distilled start it reaches 0.572 in 186 KB, within 0.035 AUC of the full-precision baseline at two orders of magnitude less memory. It is the smallest option we have, although eight bits is the more accurate one, and on an accuracy-versus-footprint view the eight-bit Micro-NAS point (0.610, 204 KB, 2.3 μJ) is the best size–accuracy trade-off while unconstrained eight-bit leads on accuracy (0.647, 790 KB).

6 Limitations

The main gap is hardware. We timed the models on the laptop, computed their sizes and RAM, and estimated energy from the Horowitz figures, but we ran nothing on the Pi 5 or the STM32H7, and the energy estimate in particular ignores the SRAM and memory traffic that tends to dominate on a Cortex-M. Eight-bit inference has a clean path to the board through ONNX and X-CUBE-AI; the one-bit path does not, since X-CUBE-AI takes binary only from Larq or QKeras, Larq is a now-archived TensorFlow library whose compute engine has no Windows build, and the Cortex-M7 has no popcount instruction, so a real 1-bit kernel falls back to software bit-counting that can eat the saving. We therefore make no speed claim for binary, and treat it only as a way to save space. Finally, a byte sequence is about five times longer than the word sequence, trading flash for on-chip memory bandwidth that only a device can settle, and all of this is cross-lingual transfer on the small MIND split.

7 Ethics and Conclusion

MIND and xMIND are used under their non-commercial research licences; the work touches no personal data beyond the public logs, and the translations are machine-generated. Replacing the word-embedding table with a byte-level encoder removes the memory wall and lets one sub-two-megabyte model read fourteen languages. Eight-bit Micro-NAS is the practical choice at 204 KB and 0.610 AUC; the binary model, once trained properly, is the fallback for the tightest memory at 0.572 and 186 KB. What is still missing is a measurement on real boards and a one-bit runtime (Larq or X-CUBE-AI on Linux), which is where we take it next. The pipeline, the ONNX encoder and the cold-start prior are released.

References

1. Clark, J.H., Garrette, D., Turc, I., Wieting, J.: CANINE: Pre-training an efficient tokenization-free encoder for language representation. In: Transactions of the ACL (2022), arXiv:2103.06874

2. Iana, A., Glavaš, G., Paulheim, H.: MIND your language: A multilingual dataset for cross-lingual news recommendation. In: Proceedings of SIGIR (2024). https://doi.org/10.1145/3626772.3657867, xMIND; arXiv:2403.17876
3. Kusupati, A., Bhatt, G., Rege, A., Wallingford, M., Sinha, A., Ramanujan, V., Howard-Snyder, W., Chen, K., Kakade, S., Jain, P., Farhadi, A.: Matryoshka representation learning. In: Proceedings of NeurIPS (2022), arXiv:2205.13147
4. Lin, J., Chen, W.M., Lin, Y., Cohn, J., Gan, C., Han, S.: MCUNet: Tiny deep learning on iot devices. In: Proceedings of NeurIPS (2020), arXiv:2007.10319
5. Lin, M., et al.: NAS-BNN: Neural architecture search for binary neural networks. Pattern Recognition (2024), arXiv:2408.15484; code: github.com/VDIGPKU/NAS-BNN
6. Liu, Z., Shen, Z., Savvides, M., Cheng, K.T.: ReActNet: Towards precise binary neural network with generalized activation functions. In: Proceedings of ECCV (2020), arXiv:2003.03488
7. Liu, Z., Wu, B., Luo, W., Yang, X., Liu, W., Cheng, K.T.: Bi-real net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm. In: Proceedings of ECCV (2018), arXiv:1808.00278
8. NLLB Team: No language left behind: Scaling human-centered machine translation (2022), arXiv:2207.04672
9. Pighetti, A., Bellotti, F., Mohammady, S., Lazzaroni, L., Ballout, H., Berta, R.: Exploring neural architecture search (NAS) for binary neural networks (BNNs) in the new WakeVision. In: Applications in Electronics Pervading Industry, Environment and Society (ApplePies), Lecture Notes in Electrical Engineering, vol. 1553 (2026). https://doi.org/10.1007/978-3-032-17174-0_12
10. Rastegari, M., Ordonez, V., Redmon, J., Farhadi, A.: XNOR-Net: Imagenet classification using binary convolutional neural networks. In: Proceedings of ECCV (2016)
11. Reimers, N., Gurevych, I.: Making monolingual sentence embeddings multilingual using knowledge distillation. In: Proceedings of EMNLP (2020), arXiv:2004.09813
12. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M.: MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In: Proceedings of NeurIPS (2020), arXiv:2002.10957
13. Wu, C., Wu, F., An, M., Huang, J., Huang, Y., Xie, X.: Neural news recommendation with attentive multi-view learning. In: Proceedings of IJCAI (2019)
14. Wu, C., Wu, F., Ge, S., Qi, T., Huang, Y., Xie, X.: Neural news recommendation with multi-head self-attention. In: Proceedings of EMNLP-IJCNLP. pp. 6389–6394 (2019)
15. Wu, C., Wu, F., Huang, Y., Xie, X.: Personalized news recommendation: Methods and challenges. ACM Transactions on Information Systems (2023), arXiv:2106.08934
16. Wu, F., et al.: MIND: A large-scale dataset for news recommendation. In: Proceedings of ACL. pp. 3597–3606 (2020)